SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
॥वसुधैव कुटुम्बकम्॥ Re-Accredited by NAAC with 'A++' Grade

Subject: Programming with Java

ASSIGNMENT No : 4

TITLE : Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Name: Arpan Gupta	Class: CSE A2
PRN: 25070122046	Batch: 2025-2029

Problem Statement

Develop a Java application demonstrating the use of wrapper classes for conversion between primitive data types and objects, and performing basic operations on wrapped values.

Learning Objectives

To perform conversion between primitive data types and wrapper objects using Java wrapper classes.

Theory

Wrapper classes are predefined classes that encapsulate primitive data types into objects. Java provides wrapper classes such as Integer, Double, Character, and Boolean for every primitive type. They support autoboxing, unboxing, parsing, and conversion methods. Wrapper classes are widely used in Java Collections Framework and when object representation of primitive values is required.

Outcome

Something > src > lab > WrapperExample.java > WrapperExample > main(String[])

```
1  package lab;
2  class WrapperExample {
       Run | Debug
3      public static void main(String[] args) {
4          int x = 5;
5          Integer wrappedX = Integer.valueOf(x);
6          int unwrappedX = wrappedX.intValue();
7          System.out.println("Wrapped: " + wrappedX + ", Unwrapped: " + unwrappedX);
8      }
9  }
```

Conclusion

This assignment demonstrates the usage of Wrapper classes, which are widely used in Java Collections Framework and when object representation of primitive values is required.

Exercise

1.Create a program to convert student marks from String format to Integer and calculate total marks.

StudentMarks.java

Something > src > lab > StudentMarks.java > StudentMarks

```java
package lab;

/**
 * StudentMarks - Demonstrates converting student marks from String format
 * to Integer objects using Wrapper class (Integer.valueOf()) and
 * calculating total marks via auto-unboxing.
 */

public class StudentMarks {
    Run | Debug
    public static void main(String[] args) {
        // Step 1: Student marks in String format (as received from input/DB)
        String[] marksInString = {"85", "90", "78", "92", "88"};

        // Step 2: Convert String marks to Integer objects using Wrapper class
        Integer[] marksInInteger = new Integer[marksInString.length];
        for (int i = 0; i < marksInString.length; i++) {
            marksInInteger[i] = Integer.valueOf(marksInString[i]); // String → Integer
        }

        // Step 3: Calculate total marks (auto-unboxing Integer → int)
        int totalMarks = 0;
        for (Integer mark : marksInInteger) {
            totalMarks += mark; // Auto-unboxing: Integer.intValue() called automatically
        }

        // Step 4: Display results
        System.out.println(x: "=== Student Marks Report ===");
        System.out.println(x: "Original String marks:");
        for (String mark : marksInString) {
            System.out.print(mark + "  ");
        }

        System.out.println(x: "\n\nConverted Integer objects:");
        for (Integer mark : marksInInteger) {
            System.out.print(mark + "  ");
        }

        System.out.println("\n\nTotal Marks (via Wrapper conversion + unboxing): " + totalMarks);

        // Bonus: Demonstrate explicit unboxing using intValue()
        System.out.println(x: "\n--- Explicit Unboxing Demo ---");
        Integer explicitInteger = Integer.valueOf(s: "97");
        int explicitInt = explicitInteger.intValue(); // Explicit unboxing
        System.out.println("String \"97\" → Integer object: " + explicitInteger);
        System.out.println("Integer object → int via intValue(): " + explicitInt);
    }
}
```

2. Develop an **Employee Payroll System** that accepts employee IDs, basic salary, and bonus amounts from the user. Convert the entered values into wrapper objects and perform validation operations to ensure valid salary values before calculating the net salary.

EmployeePayroll.java

Something > src > lab > EmployeePayroll.java > { } lab

```java
package lab;

import java.util.Scanner;

/**
 * EmployeePayroll - Demonstrates the use of Wrapper classes (Integer, Double)
 * for converting user input, validating salary values, and calculating net salary
 * with auto-unboxing and explicit unboxing.
 */

public class EmployeePayroll {
    Run | Debug
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.println("========== Employee Payroll System ==========\n");

        // ---------- Step 1: Accept Employee ID ----------
        System.out.print("Enter Employee ID: ");
        String empIdStr = scanner.nextLine();
        Integer employeeId = Integer.valueOf(empIdStr); // String → Integer wrapper
        System.out.println("✓ Employee ID (Wrapper Integer): " + employeeId);
        System.out.println("  (Using intValue() explicitly: " + employeeId.intValue() + ")\n");

        // ---------- Step 2: Accept Basic Salary ----------
        System.out.print("Enter Basic Salary: ");
        String basicSalaryStr = scanner.nextLine();
        Double basicSalary = Double.valueOf(basicSalaryStr); // String → Double wrapper

        // ---------- Step 3: Validate Basic Salary ----------
        if (basicSalary <= 0) {
            System.out.println("✗ ERROR: Basic salary must be greater than 0.");
            scanner.close();
            return;
        }
        System.out.println("✓ Basic Salary (Wrapper Double): " + basicSalary);
        System.out.println("  (Using doubleValue() explicitly: " + basicSalary.doubleValue() + ")\n");

        // ---------- Step 4: Accept Bonus Amount ----------
        System.out.print("Enter Bonus Amount: ");
        String bonusStr = scanner.nextLine();
        Double bonus = Double.valueOf(bonusStr); // String → Double wrapper

```

```
43          // ---------- Step 5: Validate Bonus ----------
44          if (bonus < 0) {
45              System.out.println(x: "✗ ERROR: Bonus amount cannot be negative.");
46              System.out.println(x: "  Setting bonus to 0.0 (default).");
47              bonus = Double.valueOf(d: 0.0);
48          }
49          System.out.println("✓ Bonus (Wrapper Double): " + bonus);
50          System.out.println("  (Using doubleValue() explicitly: " + bonus.doubleValue() + ")\n");
51
52          // ---------- Step 6: Calculate Net Salary (auto-unboxing) ----------
53          double netSalary = basicSalary + bonus; // Auto-unboxing: Double → double
54          System.out.println(x: "--- Calculation (auto-unboxing) ---");
55          System.out.println(x: "  Net Salary = Basic Salary + Bonus");
56          System.out.println("  Net Salary = " + basicSalary + " + " + bonus + " = " + netSalary + "\n");
57
58          // ---------- Step 7: Display Payroll Summary ----------
59          System.out.println(x: "========== Payroll Summary ==========");
60          System.out.println("Employee ID            : " + employeeId);
61          System.out.println("Basic Salary           : ₹ " + String.format(format: "%.2f", basicSalary));
62          System.out.println("Bonus                  : ₹ " + String.format(format: "%.2f", bonus));
63          System.out.println(x: "-------------------------------------");
64          System.out.println("Net Salary             : ₹ " + String.format(format: "%.2f", netSalary));
65          System.out.println(x: "=====================================\n");
66
67          // ---------- Step 8: Bonus Demo – Explicit Unboxing ----------
68          System.out.println(x: "--- Explicit Unboxing Demo ---");
69          int rawEmpId = employeeId.intValue();       // Integer → int
70          double rawBasic = basicSalary.doubleValue(); // Double → double
71          double rawBonus = bonus.doubleValue();       // Double → double
72          double rawNet = rawBasic + rawBonus;
73          System.out.println("Employee ID (int)          : " + rawEmpId);
74          System.out.println("Basic Salary (double)      : " + rawBasic);
75          System.out.println("Bonus (double)             : " + rawBonus);
76          System.out.println("Net Salary (double)        : " + rawNet);
77
78          scanner.close();
79      }
80  }
```